

```

'use strict';

const VERSION = '5.1.4';
// Scope isolates caches of different GitHub Pages repositories on the same origin
const PREFIX = 'aethernfall:' + self.registration.scope + ':';
const ASSET_TIER_PREFIX = 'aethernfall-assets:' + self.registration.scope;
const CACHE_NAME = PREFIX + VERSION;
const VALID_TIERS = new Set(['low', 'medium', 'high', 'very-high', 'ultra']);
const fallbackTier = 'low';
const LOW_PACKS = [
  './assets/packs/low/common-actors.webp', './assets/packs/low/common-equipment.w
  './assets/packs/low/ashfield-actors.webp', './assets/packs/low/ashfield-terrain
  './assets/packs/low/frostmere-actors.webp', './assets/packs/low/frostmere-terra
  './assets/packs/low/mistwood-actors.webp', './assets/packs/low/mistwood-terrain
  './assets/packs/low/starreach-actors.webp', './assets/packs/low/starreach-terra
  './assets/packs/low/stonevale-actors.webp', './assets/packs/low/stonevale-terra
];
const CORE = ['./assets/art/portal.svg', './debug-overlay.js?v=' + VERSION, './ph
function assetTierCacheName(tier) { return ASSET_TIER_PREFIX + VERSION + ':' + ti
async function cacheAssetTier(tier) {
  if (!VALID_TIERS.has(tier) || tier === fallbackTier) return false;
  const response = await fetch(new URL('./assets/manifest.json?v=' + VERSION, sel
  if (!response.ok) return false;
  const manifest = await response.json();
  if (manifest.buildVersion !== VERSION) return false;
  const urls = manifest.packs.filter(pack => pack.tier === tier).map(pack => new
  await (await caches.open(assetTierCacheName(tier))).addAll(urls);
  return true;
}
self.addEventListener('install', event => {
  // Reject a partial installation; the previous worker stays usable.
  event.waitUntil(caches.open(CACHE_NAME).then(cache => cache.addAll(CORE.map(url
    cache: 'reload'
  ))))));
});
self.addEventListener('message', event => {
  if (event.data?.type === 'ACTIVATE') event.waitUntil(self.skipWaiting());
  if (event.data?.type === 'CACHE_ASSET_TIER' && event.data.version === VERSION &
});
self.addEventListener('activate', event => {
  event.waitUntil(caches.keys().then(keys => Promise.all(keys.filter(key => (key.
  ));
});
self.addEventListener('fetch', event => {

```

```

const req = event.request,
  url = new URL(req.url),
  scope = new URL(self.registration.scope);
if (req.method !== 'GET' || url.origin !== scope.origin || !url.pathname.startsWith
const relative = url.pathname.slice(scope.pathname.length);
if (req.mode === 'navigate' && (relative === '' || relative === 'index.html'))
  // Serve one fully installed release, even when the network is unreliable.
  event.respondWith(caches.open(CACHE_NAME).then(async cache => (await cache.ma
} else if (relative.startsWith('assets/packs/')) {
  event.respondWith(caches.match(req).then(async cached => {
    if (cached) return cached;
    const response = await fetch(req);
    const tier = relative.split('/')[2];
    if (response.ok && VALID_TIERS.has(tier)) (await caches.open(assetTierCache
    return response;
  }));
} else if (CORE.some(path => new URL(path, scope).href === url.href)) {
  event.respondWith(caches.open(CACHE_NAME).then(async cache => (await cache.ma
}
});

```